

Algoritmos-de-busqueda-con-ejemp...



misimisi



Inteligencia Artificial



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada

QUIERES
15€?

TRAER A TU CRUSH
DE APUNTES ♡

**WUOLAH**

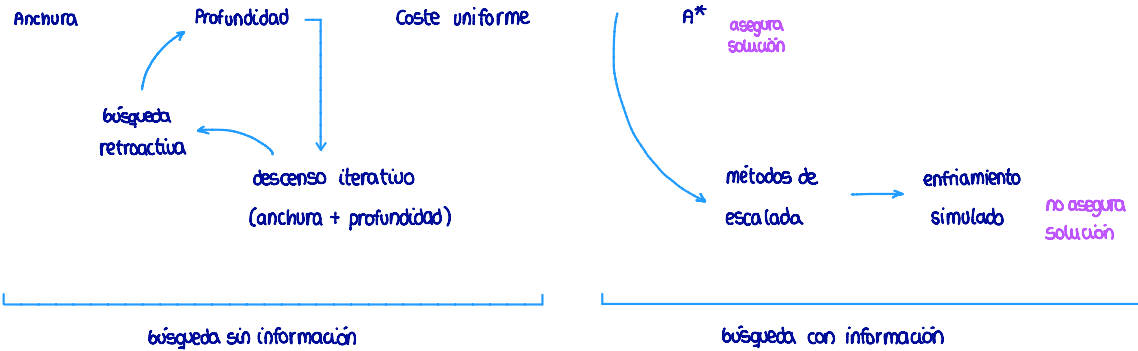
QUIERES
CONSEGUIR
15€?? →

TRÁENOS A TU
CRUSH DE APUNTES

ANTES DE QUE
LOS QUEME



Resumen de los algoritmos de búsqueda en clase de practicas



En Russell: Un enfoque moderno se llama "frontier" a ABIERTOS y "explored" a CERRADOS.

CÓMO REPRESENTAR Y RESOLVER UN PROBLEMA DE BÚSQUEDA.

- ① Representación del concepto de estado (fila, columna, orientación)
- ②
 - 1 Estado inicial (fila, columna, orientación) del agente
 - 2 Estado meta (fila, columna, ^{irrelevante} x) del objetivo
 - 3 operadores para cambiar de un estado a otro
 - ① Avanzar
 - ② Izda 90°
 - ③ Dcha 90°

Con esto ya podemos abordarlo como un problema de búsqueda.

DECISIONES QUE TOMAR AL DISEÑARLO

- ① Queremos el óptimo:
 - en acciones → anchura
 - en coste → coste uniforme o A* (con heurística admisible).
- ② No queremos óptimo: cualquier otro algoritmo (solo me interesa si hay solución).

```
struct Nodo {  
    estado st;  
    int coste;  
    list < Action > plan;  
}
```

Un nodo es un estado enriquecido. Es el estado más información adicional necesaria para encontrar el camino.



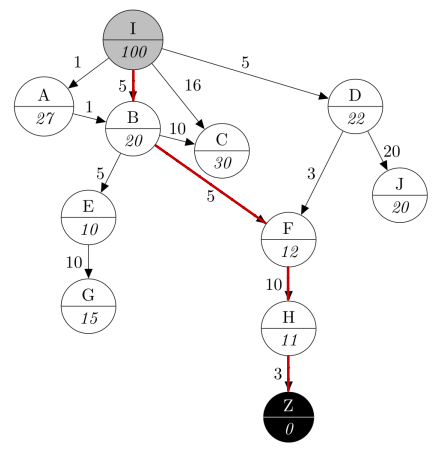
si
consigues
que suba
apuntes, te
llevas 15€ +
5 Wuolah
Coins para
los sorteos

Búsqueda en anchura

Devuelve el camino con el mínimo número de operaciones.

- * ABIERTOS es una cola.
- * No tiene en cuenta el coste de los arcos (cada arco vale 1).

la estructura de un nodo es (nombre, padre).



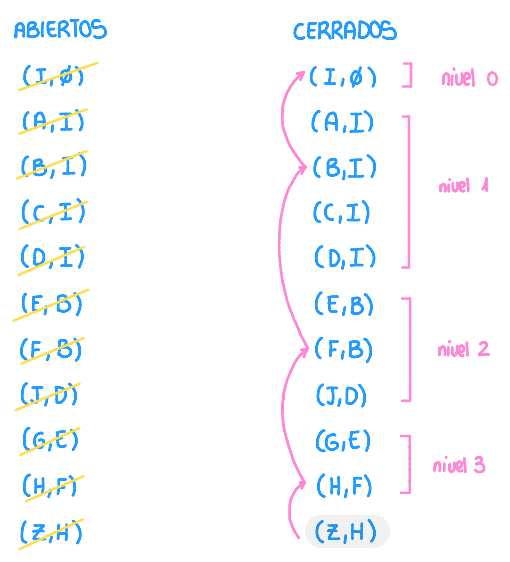
ALGORITMO

1. Colocar el estado inicial en ABIERTOS
2. Coger el primer nodo de ABIERTOS
 - 2.1 ¿ES SOLUCIÓN?
 - Sí: He terminado y construyo el camino hacia atrás, encadenando padres e hijos en la lista de cerrados.
 - No: Meto el nodo en cerrados y lo expando (meto todos sus hijos en abiertos)
 - 2.2 Vuelvo a 2

Notas importantes:

1. Si un nodo ya está en las listas, no se vuelve a meter (se omite).
2. En caso de empate, priorizar por el orden alfabético.

Proceso:



Solución:

I → B → F → H → Z.

Acuerdo:

Cuando encontremos un nodo terminal (aparece en abiertos) vamos a terminar la búsqueda porque es algo trivial.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

QUIERES 15€ ?



TRAER A TU CRUSH DE APUNTES 



si juegas con
fuego te fiegas

si consigues que suba apuntes, te llevas 15€
+ 5 Wuolah Coins para los próximos sorteos



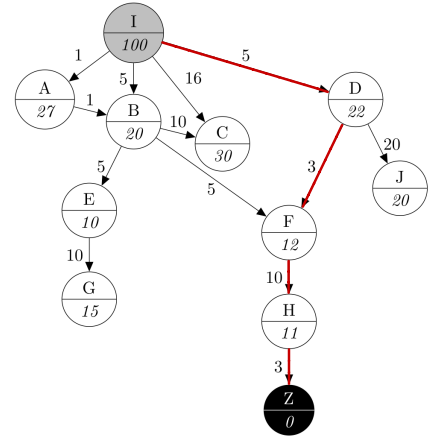
WUOLAH

Búsqueda en profundidad

Asegura llegar a la solución (si la hay).

- * ABIERTOS es una pila.*
- * No tiene en cuenta el coste de los arcos (cada arco vale 1).

la estructura de un nodo es (nombre, padre).



(*) El profe suele invertir la lista ($\downarrow$), e invertir el orden al meter los nodos ($\leftarrow$) para que salga igual que en anchura, pero da igual.

ALGORITMO

1. Colocar el estado inicial en ABIERTOS
2. Coger el último nodo de ABIERTOS
 - 2.1 ¿Es solución?
 - Sí: He terminado y construyo el camino hacia atrás, encadenando padres e hijos en la lista de cerrados.
 - No: Meto el nodo en cerrados y lo expando (meto todos sus hijos en abiertos)
 - 2.2 Vuelvo a 2

Notas importantes:

1. Si un nodo ya está en las listas, no se vuelve a meter (se omite).
2. En caso de empate, priorizar por el orden alfabético.

Proceso:

ABIERTOS	CERRADOS
(I, $\emptyset$)	(I, $\emptyset$)
(A, I)	(D, I)
(B, I)	(J, D)
(C, I)	(F, D)
(D, I)	(H, F)
(E, D)	(Z, H)
(I, D)	
(H, F)	
(Z, H)	

Solución:

I $\rightarrow$ D $\rightarrow$ F $\rightarrow$ H $\rightarrow$ Z.

QUIERES CONSEGUIR 15€??

TRÁENOS A TU CRUSH DE APUNTES

ANTES DE QUE LOS QUEME



Búsqueda con coste uniforme

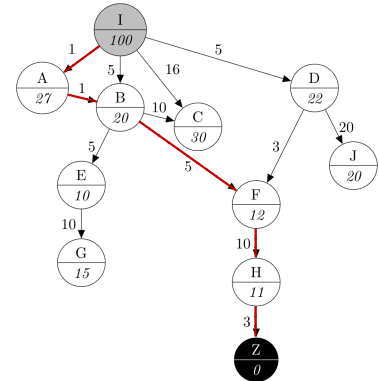
Devuelve el camino con menor coste.

- * ABIERTOS es una cola con prioridad.
- * Si tiene en cuenta el coste de los arcos.

La estructura de un nodo es (nombre, coste, padre).

El coste se calcula como el coste del padre + coste del arco (operación), y representa lo que cuesta llegar hasta ese nodo desde el nodo inicial.

DIJKSTRA



ALGORITMO

- Colocar el estado inicial en ABIERTOS.
- Coger el nodo con menos coste de la lista de ABIERTOS.
 - ¿Es solución?
 - Sí: He terminado y construyo el camino hacia atrás, encadenando padres e hijos en la lista de cerrados.
 - No: Meto el nodo en cerrados y lo expando (meto todos sus hijos en abiertos con sus respectivos costes).
 - Vuelvo a 2.

Notas importantes:

- Si un nodo ya está en las listas, nos quedamos con el que menos coste tiene y omitimos el otro.
- En caso de empate, prioritar por el orden alfabético.
- Si en la lista de cerrados los costes no ascienden en valor, no lo estamos haciendo bien.
- Si un nodo ya está en cerrados ya no lo puedo repetir.

Proceso:

ABIERTOS	CERRADOS
(I, 0, ∅)	(I, 0, ∅)
(A, 1, I)	(A, 1, I)
(B, 5, I)	(B, 2, A)
(C, 16, I)	(D, 5, J)
(D, 5, I)	(E, 7, B)
(B, 2, A)	(F, 7, B)
(C, 12, B)	(C, 12, B)
(E, 7, B)	(G, 17, E)
(F, 7, B)	(H, 17, F)
(F, 8, D)	(Z, 20, H)
(J, 25, D)	
(G, 17, E)	
(H, 17, F)	
(Z, 20, H)	

Solución:

I → A → B → F → H → Z con coste 20.



si consigues que suba apuntes, te llevas 15€ + 5 Wuolah Coins para los sorteos

A*

ABIERTOS

CERRADOS

Notación: (Nodo, g, h, f, padre)

(Nodo, g, padre)

↑ camino
↑ valor
↑ $f = g + h$ (se escoge el de menos f)

Notas importantes:

1. Si $h = 0$ (no hay heurísticas, solo caminos), es coste uniforme.
2. Es bueno ir poniendo las actualizaciones al lado y no abajo.

Proceso:

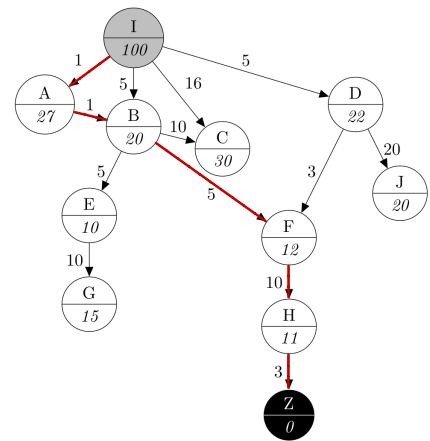
(no corregido)

ABIERTOS

(I, 0, 100, 100, ∅)
(A, 1, 27, 28, I)
(B, 5, 20, 25, I)
~~(C, 16, 30, 46, I)~~
(D, 5, 22, 27, I)
~~(E, 15, 30, 45, B)~~
(F, 10, 12, 22, B)
~~(G, 20, 15, 35, E)~~
~~(H, 20, 11, 31, F)~~
(F, 8, 12, 20, D)
(J, 25, 20, 45, D)
~~(H, 18, 11, 29, F)~~
(B, 2, 20, 22, A)
(C, 12, 30, 42, B)
(E, 7, 10, 17, B)
(F, 7, 12, 19, B)
(G, 17, 15, 32, E)
(H, 17, 11, 28, F)
(Z, 20, 0, 20, H)

CERRADOS

(I, 0, ∅)
~~(B, 5, I)~~
~~(E, 10, B)~~
~~(F, 10, B)~~
(D, 5, I)
~~(F, 8, D)~~
(A, 1, I)
(B, 2, A)
(E, 7, B)
(F, 7, B)
(H, 17, F)
(Z, 20, H)



Solución:

$I \rightarrow A \rightarrow B \rightarrow F \rightarrow H \rightarrow Z$ con coste 20.

¿Cuándo es admisible la heurística del A*?

Cuando se cumple $\forall n \ h(n) \leq h^*(n)$ (lo de dentro del nodo $\leq$ coste óptimo desde nodo n a nodo solución).

$h(H) = 11 \leq h^*(H) = 3$, luego la heurística no es admisible.

Con encontrar uno que no valga no nos sirve. Si uno sí vale, seguimos buscando.

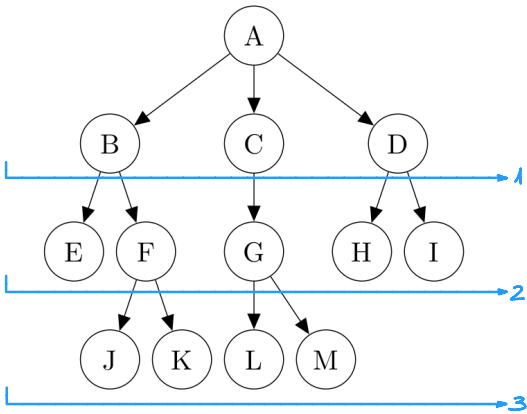
Si primero analizáramos la raíz es más fácil:

Nuestra solución es 20 ($h^*(I) = 20$) y $h(I) = 100$, luego la heurística no es admisible.

Que una heurística sea admisible nos garantiza que la solución que nos da es óptima. Si no es admisible, no garantiza la mejor solución posible.

Descenso iterativo
Búsqueda en profundidad por niveles

Se tiene que saber el grado de poda. Si no, asumimos que es 1.
Meto los hijos al reus porque es profundidad (consultar búsqueda en profundidad).
! Todo lo que está a más distancia que el grado de poda no lo tiene en cuenta.



Proceso:

ABIERTOS	CERRADOS	
(A, ∅)	(A, ∅)	nivel = 1
(D, A)	(B, A)	
(C, A)	(C, A)	
(B, A)	(D, A)	iteración 1
(A, ∅)	(A, ∅)	nivel = 2
(D, A)	(B, A)	
(C, A)	(E, B)	
(B, A)	(F, B)	
(F, B)	(C, A)	
(E, B)	(G, C)	
(G, C)	(D, A)	
(I, D)	(H, D)	
(H, D)	(I, D)	iteración 2
(A, ∅)	(A, ∅)	nivel = 3
(D, A)	(B, A)	
(C, A)	(E, B)	
(B, A)	(F, B)	
(F, B)	(J, F)	
(E, B)	(K, F)	
(K, F)	(C, A)	
(J, F)	(G, C)	
(G, C)	(L, G)	
(M, G)	(M, G)	
(L, G)	(D, A)	
(I, D)	(H, D)	
(H, D)	(I, D)	iteración 3

QUIERES
CONSEGUIR
15€?? →

TRÁENOS A TU
CRUSH DE APUNTES

ANTES DE QUE
LOS QUEME



Búsqueda retroactiva

Voy haciendo una búsqueda normal hacia abajo hasta que encuentro el nodo final.

Notas importantes:

1. Si miro todas las secuencias, los nodos con "0" definen el orden de búsqueda.

Proceso:

A^0 ← los que he explorado hijos de este nodo

$A^1 B^0$

$A^1 B^1 E^0$

$A^1 B^1$ ← no tiene hijos, lo elimino

$A^1 B^2 F^0$

$A^1 B^2 F^1 J^0$

$A^1 B^2 F^1$

$A^1 B^2 F^2 K^0$

$A^1 B^2 F^2$

$A^1 B^2$

A^1

$A^2 C^0$

$A^2 C^1 G^0$

$A^2 C^1 G^1 L^0$

$A^2 C^1 G^1$

$A^2 C^1 G^2 H^0$

$A^2 C^1 G^2$

$A^2 C^1$

A^2

$A^3 D^0$

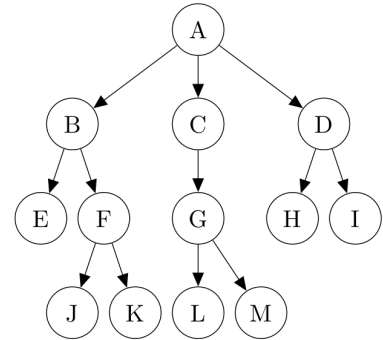
$A^3 D^1 H^0$

$A^3 D^1$

$A^3 D^2 I^0$

$A^3 D^2$

A^3

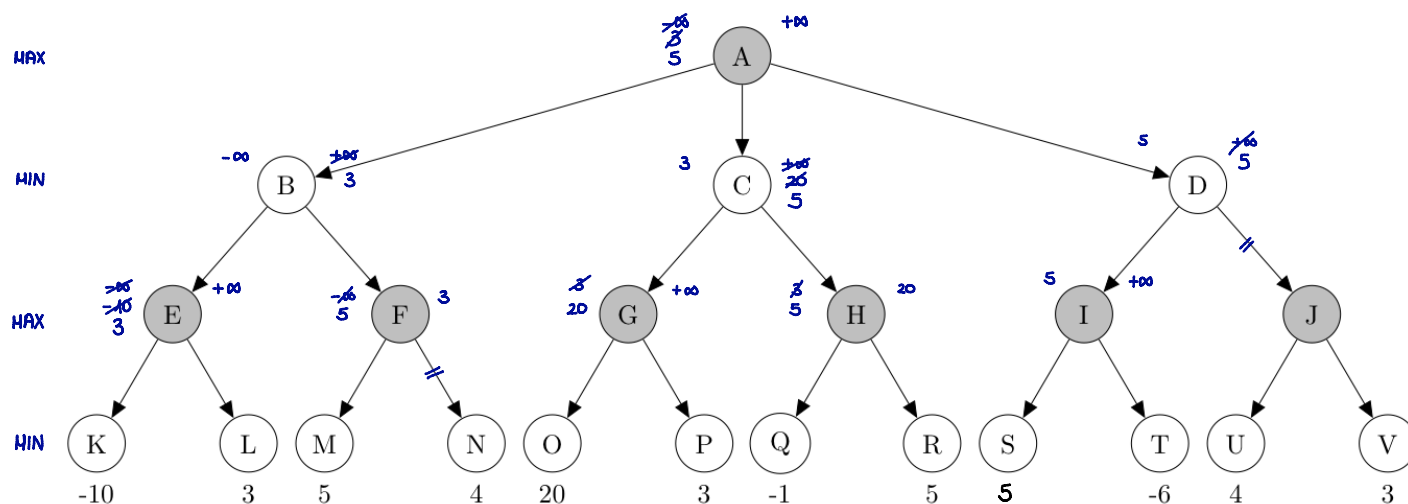


si
consigues
que suba
apuntes, te
llevas 15€ +
5 Wuolah
Coins para
los sorteos

Poda alfa-beta

Algoritmo:

1. Poner MAX y MIN en cada nivel.
2. Escribir en la raíz $\alpha = -\infty$ y $\beta = +\infty$
3. Cada vez que bajo a un nodo bajo la α y la β . Bajo hasta llegar al padre de un nodo hoja.
4. Para evaluar un nodo:
 - x Si el nodo es MAX, pongo en α (izda) el valor de su descendiente.
 - x Si el nodo es MIN, pongo en β (dcha) el valor de su descendiente.
5. Poda: Si después de evaluar un nodo $\alpha \geq \beta$, no miro los demás descendientes del nodo (podo las ramas con //).
6. Para subir las evaluaciones hacia los nodos de arriba:
 - x Si el nodo de arriba es MAX, pongo el valor máximo entre α y β en la α del padre si el nuevo α es mayor que el α actual.
 - x Si el nodo de arriba es MIN, pongo el valor mínimo entre α y β en la β del padre si el nuevo β es menor que el β actual.



7. El coste minimax:

- x Si el nodo raíz es MAX, es el α de la raíz.
- x Si el nodo raíz es MIN, es el β de la raíz.

Coste minimax = 5.